Automaten mit Ti*k*Z erstellen

Fortbildungsmaterial für MINT-Lehrkräfte

25. Mai 2025

Endliche Automaten sind fundamentale Konzepte in der theoretischen Informatik und bilden die Grundlage für das Verständnis formaler Sprachen und Berechnungsmodelle. Die visuelle Darstellung von Automaten als Zustandsdiagramme ist essentiell für das Verständnis ihrer Funktionsweise. Mit Ti*k*Z können MINT-Lehrkräfte professionelle und präzise Automatendiagramme erstellen, die den mathematischen Standards entsprechen und gleichzeitig didaktisch wertvoll sind. Diese Handreichung zeigt, wie Sie mit Ti*k*Z von einfachen bis hin zu komplexen Automaten alles darstellen können.

1 Grundlagen und benötigte Bibliotheken

Ti*k*Z bietet mit der `automata`-Bibliothek spezialisierte Funktionen für die Darstellung von Automaten. Diese Bibliothek stellt vordefinierte Stile für Zustände, Übergänge und spezielle Markierungen bereit, die den üblichen Konventionen der theoretischen Informatik entsprechen.
Für die Erstellung professioneller Automatendiagramme benötigen wir mehrere Ti*k*Z-Bibliotheken, die verschiedene Aspekte der Darstellung abdecken:

```
\usepackage{tikz}
\usetikzlibrary{automata, positioning, arrows.meta}
```

Die `automata`-Bibliothek ist das Herzstück für Automatendiagramme und stellt die Grundstile `state`, `initial` und `accepting` zur Verfügung. Zusätzlich definiert sie automatisch das Verhalten für Start- und Endzustände. Die `positioning`-Bibliothek ermöglicht die intuitive Platzierung von Zuständen relativ zueinander, was bei größeren Automaten unerlässlich ist. Mit `arrows.meta` erhalten wir moderne, saubere Pfeildarstellungen für die Übergänge zwischen Zuständen.
Die `automata`-Bibliothek bringt bereits einige praktische Voreinstellungen mit: Zustände werden automatisch als Kreise dargestellt, der Parameter `node distance` ist standardmäßig auf sinnvolle Werte gesetzt und die Pfeilspitzen sind für Automatendiagramme optimiert.

2 Einfache Zustände und Grundelemente

2.1 Grundlegende Zustandstypen

Ein Automat besteht aus verschiedenen Zustandstypen, die in Ti*k*Z durch spezielle Stile definiert werden:

```
\begin{tikzpicture}
  % Normaler Zustand
  \node[state] (q0) {$q_0$};

  % Startzustand
  \node[state, initial, right=3cm of q0] (q1) {$q_1$};

  % Endzustand
  \node[state, accepting, right=2cm of q1] (q2) {$q_2$};

  % Start- und Endzustand
  \node[state, initial, accepting, right=3cm of q2] (q3) {$q_3$};
\end{tikzpicture}
```

Ausgabe



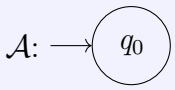
Der `state`-Stil erzeugt automatisch einen Kreis mit der entsprechenden Beschriftung. Das `initial`-Attribut fügt einen Pfeil von links hinzu, der den Startzustand kennzeichnet. Das `accepting`-Attribut erstellt den charakteristischen doppelten Kreis für Endzustände.

2.2 Automatenbeschriftung

Automaten können mit einer Beschriftung versehen werden, die typischerweise links neben dem Startzustand platziert wird:

```
\begin{tikzpicture}[initial text={$\mathcal{A}$:}]
  \node[state, initial] (q0) {$q_0$};
\end{tikzpicture}
```

Ausgabe



3 Übergänge zwischen Zuständen

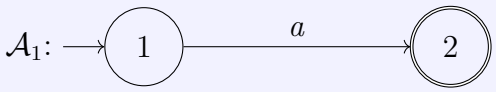
3.1 Einfache Übergänge

Übergänge werden mit dem `\draw`-Befehl und entsprechenden Pfeilnotationen erstellt:

```
\begin{tikzpicture}[initial text={$\mathcal{A}_1$:}]
  \node[state, initial] (q1) {$1$};
  \node[state, accepting, right=3cm of q1] (q2) {$2$};

  \draw[->] (q1) edge[above] node{$a$} (q2);
\end{tikzpicture}
```

Ausgabe



3.2 Mehrere Übergänge und Schleifen

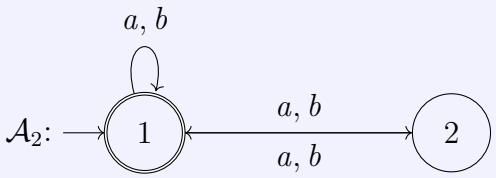
Für Übergänge mit mehreren Symbolen und Schleifen bietet Ti*k*Z verschiedene Optionen:

```
\begin{tikzpicture}[initial text={$\mathcal{A}_2$:}]
  \node[state, initial, accepting] (q1) {$1$};
  \node[state, right=3cm of q1] (q2) {$2$};

  % Schleife
  \draw[->] (q1) edge[loop above] node{$a,b$} (q1);

  % Hin- und Rueckweg
  \draw[->] (q1) edge[above] node{$a,b$} (q2);
  \draw[->] (q2) edge[below] node{$a,b$} (q1);
\end{tikzpicture}
```

Ausgabe



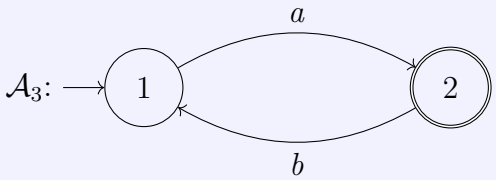
3.3 Gebogene Übergänge

Für übersichtlichere Darstellungen können Übergänge gebogen werden:

```
\begin{tikzpicture}[initial text={$\mathcal{A}_3$:}]
  \node[state, initial] (q1) {$1$};
  \node[state, accepting, right=3cm of q1] (q2) {$2$};

  % Gebogene Uebergaenge
  \draw[->] (q1) edge[bend left, above] node{$a$} (q2);
  \draw[->] (q2) edge[bend left, below] node{$b$} (q1);
\end{tikzpicture}
```

Ausgabe



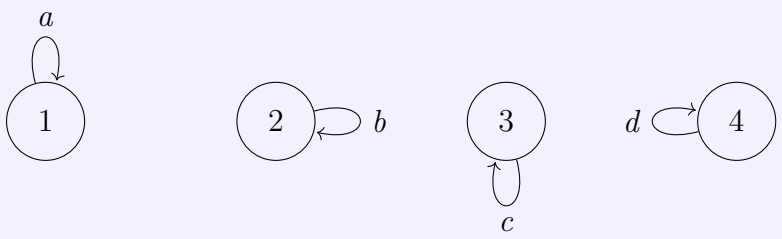
4 Verschiedene Schleifentypen

Schleifen können in verschiedene Richtungen zeigen, je nach Layout des Automaten:

```
\begin{tikzpicture}
  \node[state] (q1) {$1$};
  \node[state, right=2cm of q1] (q2) {$2$};
  \node[state, right=2cm of q2] (q3) {$3$};
  \node[state, right=2cm of q3] (q4) {$4$};

  % Verschiedene Schleifenpositionen
  \draw[->] (q1) edge[loop above] node{$a$} (q1);
  \draw[->] (q2) edge[loop right] node{$b$} (q2);
  \draw[->] (q3) edge[loop below] node{$c$} (q3);
  \draw[->] (q4) edge[loop left] node{$d$} (q4);
\end{tikzpicture}
```

Ausgabe



5 Vollständige Beispiele aus der Praxis

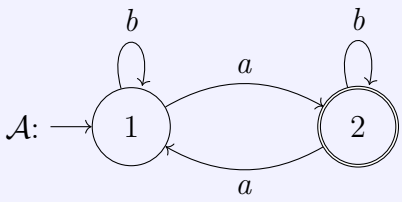
5.1 Automat für ungerade Anzahl von a's

Hier ist ein praktisches Beispiel für einen Automaten, der Wörter mit ungerader Anzahl von a's akzeptiert:

```
\begin{tikzpicture}[initial text={$\mathcal{A}$:}, node distance=3cm]
  \node[state, initial] (q1) {$1$};
  \node[state, accepting, right of=q1] (q2) {$2$};

  \draw[->] (q1) edge[bend left, above] node{$a$} (q2);
  \draw[->] (q1) edge[loop above] node{$b$} (q1);
  \draw[->] (q2) edge[bend left, below] node{$a$} (q1);
  \draw[->] (q2) edge[loop above] node{$b$} (q2);
\end{tikzpicture}
```

Ausgabe



5.2 Automat für Wörter mit Teilwort „ab“

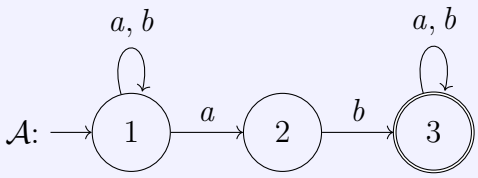
Ein etwas komplexerer Automat, der alle Wörter akzeptiert, die das Teilwort ab enthalten:

```
\begin{tikzpicture}[initial text={$\mathcal{A}$:}, node distance=2cm]
  \node[state, initial] (q1) {$1$};
```

```
  \node[state, right of=q1] (q2) {$2$};
  \node[state, accepting, right of=q2] (q3) {$3$};

  \draw[->] (q1) edge[loop above] node{$a,b$} (q1);
  \draw[->] (q1) edge[above] node{$a$} (q2);
  \draw[->] (q2) edge[above] node{$b$} (q3);
  \draw[->] (q3) edge[loop above] node{$a,b$} (q3);
\end{tikzpicture}
```

Ausgabe



6 Komplexe Automaten

6.1 Größere Automaten strukturieren

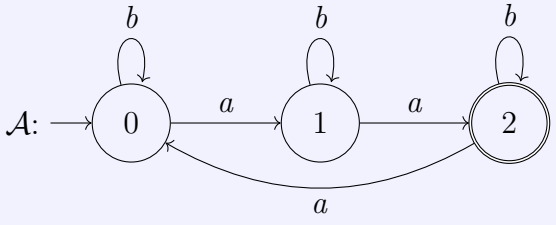
Bei komplexeren Automaten ist die Strukturierung besonders wichtig. Hier ein Beispiel eines Automaten zur Modulo-3-Rechnung:

```
\begin{tikzpicture}[initial text={$\mathcal{A}$:}, node distance=2.5cm]
  \node[state, initial] (q0) {$0$};
  \node[state, right of=q0] (q1) {$1$};
  \node[state, accepting, right of=q1] (q2) {$2$};

  % Uebergaenge fuer a (Modulo 3)
  \draw[->] (q0) edge[above] node{$a$} (q1);
  \draw[->] (q1) edge[above] node{$a$} (q2);
  \draw[->] (q2) edge[bend left, below] node{$a$} (q0);

  % Schleifen fuer b
  \draw[->] (q0) edge[loop above] node{$b$} (q0);
  \draw[->] (q1) edge[loop above] node{$b$} (q1);
  \draw[->] (q2) edge[loop above] node{$b$} (q2);
\end{tikzpicture}
```

Ausgabe



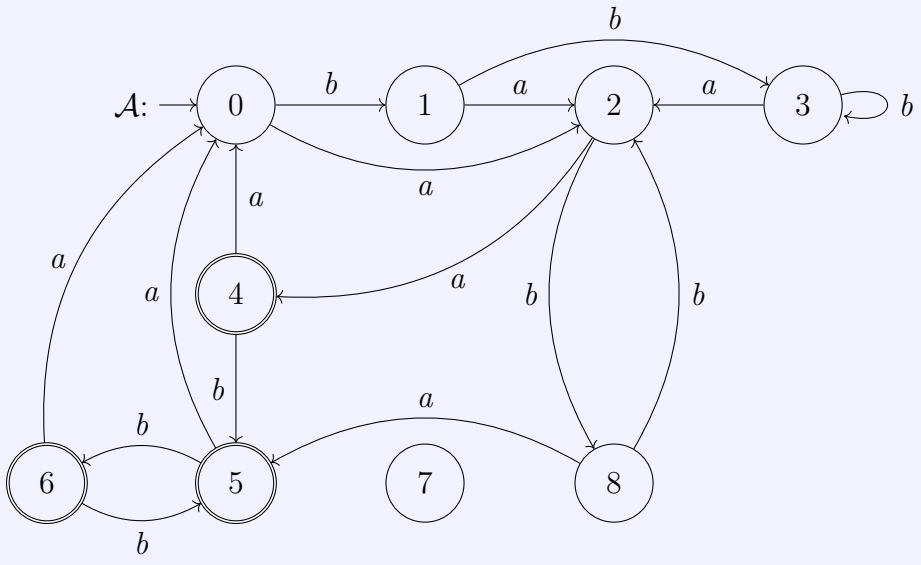
6.2 Sehr komplexer Automat

Hier ist ein Beispiel eines komplexeren Automaten mit mehreren Endzuständen und verschiedenen Übergängen:

```
\begin{tikzpicture}[initial text={$\mathcal{A}$:}, node distance=2.5cm, scale=0.9]
  \node[state, initial] (q0) {$0$};
  \node[state, right of=q0] (q1) {$1$};
  \node[state, right of=q1] (q2) {$2$};
  \node[state, right of=q2] (q3) {$3$};
  \node[state, accepting, below of=q0] (q4) {$4$};
  \node[state, accepting, below of=q4] (q5) {$5$};
  \node[state, accepting, left of=q5] (q6) {$6$};
  \node[state, right of=q5] (q7) {$7$};
  \node[state, right of=q7] (q8) {$8$};

  % Uebergaenge
  \draw[->] (q0) edge[above] node{$b$} (q1);
  \draw[->] (q0) edge[bend right, below] node{$a$} (q2);
  \draw[->] (q1) edge[above] node{$a$} (q2);
  \draw[->] (q1) edge[bend left, above] node{$b$} (q3);
  \draw[->] (q2) edge[bend left, below] node{$a$} (q4);
  \draw[->] (q2) edge[bend right, left] node{$b$} (q8);
  \draw[->] (q3) edge[above] node{$a$} (q2);
  \draw[->] (q3) edge[loop right] node{$b$} (q3);
  \draw[->] (q4) edge[right] node{$a$} (q0);
  \draw[->] (q4) edge[left] node{$b$} (q5);
  \draw[->] (q5) edge[bend right, above] node{$b$} (q6);
  \draw[->] (q5) edge[bend left, left] node{$a$} (q0);
  \draw[->] (q6) edge[bend right, below] node{$b$} (q5);
  \draw[->] (q6) edge[bend left, left] node{$a$} (q0);
  \draw[->] (q8) edge[bend right, above] node{$a$} (q5);
  \draw[->] (q8) edge[bend right, right] node{$b$} (q2);
\end{tikzpicture}
```

Ausgabe



7 Tipps für professionelle Darstellung

Die Qualität von Automatendiagrammen hängt wesentlich von einheitlichen Gestaltungsprinzipien ab. Bei größeren Automaten sollten Sie konsistente Abstände verwenden, indem Sie den Parameter `node distance` global für das gesamte Diagramm festlegen. Die Option `auto` sorgt dabei für automatische Positionierung der Übergangsbeschriftungen, wodurch Überlappungen vermieden

werden.
Für sehr große Automaten kann eine proportionale Skalierung hilfreich sein. Mit `scale=0.8` wird die gesamte Zeichnung verkleinert, während `every node/.style={scale=0.8}` zusätzlich alle Beschriftungen entsprechend anpasst. Dies gewährleistet, dass auch bei verkleinerter Darstellung alle Elemente lesbar bleiben.
Besonders wichtig ist eine konsistente mathematische Notation. Verwenden Sie einheitliche Konventionen für Zustandsnamen (z.B. $q_0, q_1, \ldots$ oder $s_0, s_1, \ldots$) und Übergangsbeschriftungen. Mehrere Symbole werden durch Kommas getrennt (a, b), während Epsilon-Übergänge mit ε notiert werden.

```
\begin{tikzpicture}[node distance=3cm, auto, scale=0.8, every node/.style={scale=0.8}]
  % Fuer Zustaende: einheitliche Notation
  \node[state] (q0) {$q_0$};
  \node[state, right of=q0] (q1) {$q_0$};
  \node[state, right of=q1] (q2) {$q_0$};

  % Fuer Uebergaenge: konsistente Beschriftung
  \draw[->] (q0) edge[above] node{$a,b$} (q1);            % Mehrere Symbole
  \draw[->] (q1) edge[above] node{$\varepsilon$} (q2);    % Epsilon-Uebergaenge
\end{tikzpicture}
```

8 Zusammenfassung

Die Erstellung von Automatendiagrammen mit Ti*k*Z bietet maximale Kontrolle über das Erscheinungsbild und gewährleistet professionelle Ergebnisse. Die `automata`-Bibliothek stellt alle wesentlichen Grundbausteine zur Verfügung, während die Kombination mit anderen Ti*k*Z-Funktionen beliebige Anpassungen ermöglicht.
Die wichtigsten Vorteile auf einen Blick: Die `automata`-Bibliothek bietet spezialisierte Stile für alle Automatenelemente. Intuitive Positionierung durch die `positioning`-Bibliothek erleichtert das Layout. Flexible Übergangsmöglichkeiten mit Schleifen, Biegungen und Beschriftungen decken alle Anforderungen ab. Konsistente Formatierung durch zentrale Stilkontrolle sorgt für professionelle Ergebnisse.
Mit den hier vorgestellten Techniken können Sie alle Arten von endlichen Automaten darstellen – von einfachen Zwei-Zustand-Automaten bis hin zu komplexen Minimierungsbeispielen. Die Kombination aus mathematischer Präzision und visueller Klarheit macht Ti*k*Z zur idealen Wahl für die Darstellung formaler Modelle in der theoretischen Informatik.